



COURSEWARE

Agentic AI Applications with Codex

Learner Guide

Course Code	C695
Version	v1.0
Release Date	20 September 2026
Training Provider	Tertiary Infotech Academy Pte. Ltd. (UEN 201200696W)
Trainer	Dr Alfred Ang

Controlled document. Verify the version before use.

Document Control

Document	Version	Date	Owner	Change summary
Learner Guide	1.0	20 September 2026	Tertiary Infotech Academy Pte. Ltd.	Course identity updated to C695; detailed concepts and ten lab guides retained

Contents

Right-click and update this field to refresh the contents.

1. Course Overview

This Learner Guide supports an end-to-end, evidence-led Codex workflow. Learners analyse the suitability of an agentic application, frame an execution contract, configure repository instructions and permissions, apply tools and reusable extensions, then evaluate correctness, reliability, efficiency, risk and human accountability. The detailed procedures are intentionally kept here and out of the visual slide deck.

SAFETY BOUNDARY

All lab data are synthetic. Work only in the supplied lab or a temporary copy. Never place live keys, personal data or proprietary secrets in prompts, files, logs or screenshots; keep destructive and external actions out of scope.

1.1 Learning Outcomes

- LO1 Analyse agentic AI applications, their strengths, limitations and suitability for a work problem.
- LO2 Correlate agent-control design with algorithm efficiency and evaluate the resulting process improvement.
- LO3 Assess methods for evaluating the effectiveness, safety and reliability of Codex-enabled workflows.
- LO4 Evaluate comparative effectiveness and recommend a governed Codex implementation using observable evidence.

1.3 End-to-End Reference Architecture

1. Select the Codex surface by repository location, duration, isolation, collaboration and policy constraints.
2. Inspect applicable AGENTS.md files, repository structure, runtime, tests and working-tree state before editing.
3. Write an execution contract that names the outcome, context, protected boundaries and observable definition of done.
4. Decompose the task, isolate independent work when useful, and keep filesystem, network and approval permissions narrow.
5. Apply the smallest causal patch, review the complete diff, and run focused plus regression checks.
6. Capture browser, terminal, test and repository evidence that proves the requested behavior rather than accepting an agent assertion.
7. Compare the accepted result with a measured baseline and issue a bounded PILOT, EXPAND, HOLD or STOP recommendation.

2. Codex Surfaces, Agentic Loops and Applicability

Analyse where Codex fits, how a task travels through the agentic loop, and where human judgment remains essential.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

2.1 The Codex product surface

Mechanism

1. Select surface
2. Provide repository
3. Bound task
4. Review result

surface	repo	goal	diff
Required state field for select surface	Required state field for provide repository	Required state field for bound task	Required state field for review result

Evidence and control

MEASURE

Task-to-surface fit score across local, IDE, cloud and app

Failure mode: A surface is selected for novelty rather than the work constraint.

Control: Choose by data location, duration, isolation and collaboration need.

Acceptance evidence: Decision matrix plus selected surface rationale

2.2 Agentic task lifecycle

Mechanism

1. Observe state
2. Form plan
3. Act with tools
4. Verify outcome

state	plan	action	verification
Required state field for observe state	Required state field for form plan	Required state field for act with tools	Required state field for verify outcome

Evidence and control

MEASURE

Verified task transitions / total task transitions

Failure mode: The agent reports completion without validating the changed system.

Control: Make verification an explicit finish condition.

Acceptance evidence: Plan, command trace, test result and final diff

2.3 Prompt as an execution contract

Mechanism

1. State outcome
2. Add context
3. Set constraints
4. Define done

objective	context	constraints	acceptance
Required state field for state outcome	Required state field for add context	Required state field for set constraints	Required state field for define done

Evidence and control

MEASURE

Acceptance criteria satisfied on first review

Failure mode: A vague request produces a broad or misdirected change.

Control: Require observable acceptance checks and protected boundaries.

Acceptance evidence: Prompt contract linked to passing checks

2.4 Repository reconnaissance

Mechanism

1. Read instructions
2. Map files
3. Inspect tests
4. Choose scope

AGENTS	tree	tests	scope
Required state field for read instructions	Required state field for map files	Required state field for inspect tests	Required state field for choose scope

Evidence and control

MEASURE

Relevant files inspected before first edit

Failure mode: Editing begins before local conventions or dependencies are understood.

Control: Front-load read-only inspection and instruction discovery.

Acceptance evidence: Inspection log and scoped file list

2.5 Tool-mediated reasoning

Mechanism

1. Choose tool
2. Execute action
3. Read output
4. Update plan

tool	command	result	next_step
Required state field for choose tool	Required state field for execute action	Required state field for read output	Required state field for update plan

Evidence and control

MEASURE

Useful tool results / total tool calls

Failure mode: Repeated tool calls add noise without reducing uncertainty.

Control: Tie each call to one decision and stop when evidence is sufficient.

Acceptance evidence: Tool result cited in the next decision

2.6 Local versus cloud execution

Mechanism

1. Locate data
2. Assess duration
3. Choose isolation
4. Dispatch task

data_zone	runtime	isolation	destination
Required state field for locate data	Required state field for assess duration	Required state field for choose isolation	Required state field for dispatch task

Evidence and control

MEASURE

Policy-compliant runs completed without relocation

Failure mode: Sensitive or stateful work is sent to an unsuitable environment.

Control: Select the execution boundary before attaching the repository.

Acceptance evidence: Environment choice and policy check

2.7 Parallel task decomposition

Mechanism

1. Find seams
2. Assign scopes
3. Run independently
4. Integrate outputs

dependency	owner	worktree	merge_order
Required state field for find seams	Required state field for assign scopes	Required state field for run independently	Required state field for integrate outputs

Evidence and control

MEASURE

Parallel speed-up with zero overlapping edits

Failure mode: Agents modify the same files or depend on unfinished shared state.

Control: Decompose only at dependency-light boundaries.

Acceptance evidence: Scope map, separate worktrees and integration checks

2.8 Strengths of coding agents

Mechanism

1. Search broadly
2. Edit consistently
3. Run checks
4. Summarise evidence

coverage	patch	tests	summary
Required state field for search broadly	Required state field for edit consistently	Required state field for run checks	Required state field for summarise evidence

Evidence and control

MEASURE

Cycle time reduction at unchanged defect escape rate

Failure mode: Speed is treated as quality despite weak review evidence.

Control: Measure throughput and quality together.

Acceptance evidence: Baseline versus assisted task metrics

2.9 Limitations and uncertainty

Mechanism

1. Detect ambiguity
2. Expose assumption
3. Seek evidence
4. Escalate choice

unknown	assumption	evidence	decision_owner
Required state field for detect ambiguity	Required state field for expose assumption	Required state field for seek evidence	Required state field for escalate choice

Evidence and control

MEASURE

Material assumptions surfaced before mutation

Failure mode: The agent silently invents project facts or permissions.

Control: Use uncertainty thresholds and explicit escalation points.

Acceptance evidence: Assumption register with resolved evidence

2.10 Human accountability boundary

Mechanism

1. Classify consequence
2. Assign owner
3. Prepare evidence
4. Approve action

risk_class	owner	evidence_pack	approval
Required state field for classify consequence	Required state field for assign owner	Required state field for prepare evidence	Required state field for approve action

Evidence and control

MEASURE

Consequential actions with named approval

Failure mode: Automation crosses an external or destructive boundary unattended.

Control: Require current evidence and a named human before consequence.

Acceptance evidence: Approval record bound to the reviewed change

2.11 Applicability decision matrix

Mechanism

1. Define problem
2. Score fit
3. Test constraints
4. Recommend posture

problem	fit_score	constraints	recommendation
Required state field for define problem	Required state field for score fit	Required state field for test constraints	Required state field for recommend posture

Evidence and control

MEASURE

Recommendation traceability to scored criteria

Failure mode: A showcase task is mistaken for a production-ready use case.

Control: Score repeatability, verifiability, risk and integration cost.

Acceptance evidence: Weighted matrix and recommendation

3. Harness Design, Context, Sandbox and Process Efficiency

Design a bounded Codex harness that turns project context, permissions and Git isolation into an efficient process.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

3.1 AGENTS.md instruction hierarchy

Mechanism

1. Discover files
2. Resolve scope
3. Apply nearest rule
4. Record compliance

root_rule	nested_rule	scope	evidence
Required state field for discover files	Required state field for resolve scope	Required state field for apply nearest rule	Required state field for record compliance

Evidence and control

MEASURE

Edits compliant with the nearest applicable instruction

Failure mode: A global rule overrides a more specific directory contract.

Control: Resolve instructions from repository root to target file.

Acceptance evidence: Instruction chain and compliant patch

3.2 Context budget engineering

Mechanism

1. Load essentials
2. Summarise findings
3. Discard noise
4. Refresh evidence

essential	summary	noise	refresh
Required state field for load essentials	Required state field for summarise findings	Required state field for discard noise	Required state field for refresh evidence

Evidence and control

MEASURE

Decision-relevant context / total loaded context

Failure mode: Large logs and irrelevant files crowd out constraints.

Control: Load progressively and summarise before expanding scope.

Acceptance evidence: Compact context ledger

3.3 Planning and checkpoints

Mechanism

1. Decompose outcome
2. Order dependencies
3. Mark checkpoint
4. Re-plan from evidence

steps	dependencies	checkpoint	status
Required state field for decompose outcome	Required state field for order dependencies	Required state field for mark checkpoint	Required state field for re-plan from evidence

Evidence and control

MEASURE

Completed checkpoints with verification

Failure mode: A long task drifts because the plan never incorporates new evidence.

Control: Update the plan at material state changes.

Acceptance evidence: Plan history and checkpoint result

3.4 Configuration layers

Mechanism

1. Read defaults
2. Apply user config
3. Apply project config
4. Confirm effective value

default	user	project	effective
Required state field for read defaults	Required state field for apply user config	Required state field for apply project config	Required state field for confirm effective value

Evidence and control

MEASURE

Configuration values with known provenance

Failure mode: A hidden configuration override changes model or permission behavior.

Control: Expose configuration precedence and validate the effective state.

Acceptance evidence: Resolved configuration table

3.5 Sandbox policy

Mechanism

1. Classify resource
2. Set access
3. Execute bounded action
4. Audit result

resource	permission	action	audit
Required state field for classify resource	Required state field for set access	Required state field for execute bounded action	Required state field for audit result

Evidence and control

MEASURE

Writes contained within the approved boundary

Failure mode: A command modifies data outside the learner or project scope.

Control: Use the narrowest filesystem and network permissions that work.

Acceptance evidence: Sandbox policy and audited path list

3.6 Approval design

Mechanism

1. Detect boundary
2. Explain command
3. Request decision
4. Resume or stop

boundary	justification	decision	result
Required state field for detect boundary	Required state field for explain command	Required state field for request decision	Required state field for resume or stop

Evidence and control

MEASURE

Approval prompts that identify exact consequence

Failure mode: Repeated or vague approvals train users to click through.

Control: Ask only at real boundaries with exact targets and effects.

Acceptance evidence: Approval decision tied to command and output

3.7 Git worktree isolation

Mechanism

1. Select base
2. Create worktree
3. Edit independently
4. Integrate commit

base_sha	branch	path	commit
Required state field for select base	Required state field for create worktree	Required state field for edit independently	Required state field for integrate commit

Evidence and control

MEASURE

Parallel tasks integrated without working-tree collision

Failure mode: Concurrent changes share one checkout and overwrite state.

Control: Allocate one worktree and branch per independent mutation stream.

Acceptance evidence: Distinct paths, branches and commit SHAs

3.8 Patch discipline

Mechanism

1. Inspect target
2. Make minimal edit
3. Review diff
4. Run focused checks

target	patch	diff	checks
Required state field for inspect target	Required state field for make minimal edit	Required state field for review diff	Required state field for run focused checks

Evidence and control

MEASURE

Changed lines that directly serve acceptance criteria

Failure mode: Broad rewrites obscure defects and overwrite user changes.

Control: Prefer minimal, reviewable patches and preserve unrelated state.

Acceptance evidence: Scoped diff and focused test output

3.9 Command safety

Mechanism

1. Resolve target
2. Quote arguments
3. Run non-destructive check
4. Perform action

target	arguments	preflight	action
Required state field for resolve target	Required state field for quote arguments	Required state field for run non-destructive check	Required state field for perform action

Evidence and control

MEASURE

Potentially destructive commands preceded by exact target checks

Failure mode: Globs, substitutions or broad paths expand unexpectedly.

Control: Resolve explicit targets and prefer recoverable operations.

Acceptance evidence: Preflight listing and action log

3.10 Process baseline

Mechanism

1. Map current flow
2. Measure delay
3. Locate rework
4. Select intervention

steps	cycle_time	rework	intervention
Required state field for map current flow	Required state field for measure delay	Required state field for locate rework	Required state field for select intervention

Evidence and control

MEASURE

Baseline cycle time and defect rate captured

Failure mode: Improvement is claimed without a pre-assistance baseline.

Control: Measure the existing process before introducing Codex.

Acceptance evidence: Baseline map and observed metrics

3.11 Efficiency without control loss

Mechanism

1. Automate repeatable step
2. Keep gate
3. Measure result
4. Adjust scope

automation	gate	metric	scope
Required state field for automate repeatable step	Required state field for keep gate	Required state field for measure result	Required state field for adjust scope

Evidence and control

MEASURE

Cycle-time gain with equal or better verification coverage

Failure mode: Efficiency removes the review step that detected material defects.

Control: Automate preparation and checking while retaining consequential gates.

Acceptance evidence: Before/after measures and unchanged control map

4. Tools, Skills, MCP, Subagents and Cloud Automation

Apply the Codex extension surface to implement repeatable, observable and portable agent workflows.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

4.1 Terminal and file tools

Mechanism

1. Inspect file
2. Choose operation
3. Apply change
4. Verify bytes

path	operation	patch	checksum
Required state field for inspect file	Required state field for choose operation	Required state field for apply change	Required state field for verify bytes

Evidence and control

MEASURE

File operations with explicit before-and-after evidence

Failure mode: Generated content bypasses project conventions or corrupts unrelated files.

Control: Use targeted search, patch and verification tools.

Acceptance evidence: Diff plus test or checksum

4.2 Browser-based verification

Mechanism

1. Start app
2. Navigate flow
3. Observe state
4. Capture defect

url	action	state	screenshot
Required state field for start app	Required state field for navigate flow	Required state field for observe state	Required state field for capture defect

Evidence and control

MEASURE

Critical user paths verified at target viewport

Failure mode: Only source code is checked; visible behavior remains broken.

Control: Pair implementation tests with real-browser evidence.

Acceptance evidence: Browser trace and screenshot

4.3 Model Context Protocol

Mechanism

1. Select server
2. Declare capability
3. Call typed tool
4. Validate response

server	schema	request	response
Required state field for select server	Required state field for declare capability	Required state field for call typed tool	Required state field for validate response

Evidence and control

MEASURE

MCP calls returning schema-valid results

Failure mode: An over-privileged server exposes unnecessary data or actions.

Control: Install only required servers and constrain their permissions.

Acceptance evidence: Server inventory, call trace and validation

4.4 Plugins and packaged capability

Mechanism

1. Inspect bundle
2. Check permissions
3. Enable component
4. Test boundary

manifest	permissions	component	test
Required state field for inspect bundle	Required state field for check permissions	Required state field for enable component	Required state field for test boundary

Evidence and control

MEASURE

Enabled capabilities with reviewed dependencies

Failure mode: A plugin is trusted solely because it is convenient.

Control: Review source, permissions and dependencies before enabling.

Acceptance evidence: Manifest review and bounded test

4.5 Reusable skills

Mechanism

1. Write trigger
2. Encode workflow
3. Add resources
4. Test near-miss

description	instructions	resources	evaluation
Required state field for write trigger	Required state field for encode workflow	Required state field for add resources	Required state field for test near-miss

Evidence and control

MEASURE

Trigger precision across positive and near-miss prompts

Failure mode: A broad description hijacks unrelated tasks.

Control: Use narrow triggers and verify both trigger and non-trigger cases.

Acceptance evidence: Skill package plus evaluation record

4.6 Subagent delegation

Mechanism

1. Define bounded task
2. Fork context
3. Work independently
4. Return evidence

task	context	output	evidence
Required state field for define bounded task	Required state field for fork context	Required state field for work independently	Required state field for return evidence

Evidence and control

MEASURE

Delegated tasks completed without scope overlap

Failure mode: The subagent lacks required context or edits shared files unexpectedly.

Control: Delegate concrete, dependency-light work with explicit deliverables.

Acceptance evidence: Task brief and returned evidence

4.7 Hooks and lifecycle checks

Mechanism

1. Detect event
2. Run policy
3. Return status
4. Block or continue

event	policy	status	decision
Required state field for detect event	Required state field for run policy	Required state field for return status	Required state field for block or continue

Evidence and control

MEASURE

Policy violations blocked before completion

Failure mode: A hook is noisy, slow or silently ignored.

Control: Keep hooks deterministic, fast and fail-closed for material rules.

Acceptance evidence: Hook output and blocked test case

4.8 Headless codex exec

Mechanism

1. Provide task
2. Select output mode
3. Stream events
4. Exit by status

prompt	jsonl	events	exit_code
Required state field for provide task	Required state field for select output mode	Required state field for stream events	Required state field for exit by status

Evidence and control

MEASURE

Machine-readable runs with reliable non-zero failure status

Failure mode: Human-oriented output is parsed as if it were a stable API.

Control: Use structured JSON events and explicit finish conditions.

Acceptance evidence: JSONL event log and process exit code

4.9 Cloud environments

Mechanism

1. Connect repository
2. Configure setup
3. Run isolated task
4. Review diff

repo	environment	task	diff
Required state field for connect repository	Required state field for configure setup	Required state field for run isolated task	Required state field for review diff

Evidence and control

MEASURE

Cloud tasks reproducible from declared setup

Failure mode: Undeclared dependencies make remote results non-reproducible.

Control: Pin setup steps, environment variables and safe secrets.

Acceptance evidence: Environment config and cloud task summary

4.10 Background and scheduled work

Mechanism

1. Define cadence
2. Select project
3. Run task
4. Route result

schedule	project	run	notification
Required state field for define cadence	Required state field for select project	Required state field for run task	Required state field for route result

Evidence and control

MEASURE

Scheduled runs producing actionable, non-duplicate results

Failure mode: Automation creates noisy tasks or mutates state without review.

Control: Use bounded prompts, idempotency and appropriate notification policy.

Acceptance evidence: Run history and action decision

4.11 Portable agent workflow

Mechanism

1. Declare inputs
2. Bundle instructions
3. Run across surface
4. Compare outputs

inputs	AGENTS	surface	comparison
Required state field for declare inputs	Required state field for bundle instructions	Required state field for run across surface	Required state field for compare outputs

Evidence and control

MEASURE

Equivalent acceptance results across supported surfaces

Failure mode: The workflow depends on hidden local state.

Control: Move requirements into versioned repository artifacts.

Acceptance evidence: Portable repository and cross-surface comparison

5. Evaluation, Comparative Effectiveness and Governed Adoption

Evaluate Codex workflows with golden tasks, risk-adjusted measures and a defensible adoption recommendation.

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

5.1 Evaluation question design

Mechanism

1. Name decision
2. Choose criterion
3. Define measure
4. Set threshold

decision	criterion	measure	threshold
Required state field for name decision	Required state field for choose criterion	Required state field for define measure	Required state field for set threshold

Evidence and control

MEASURE

Criteria with decision-linked thresholds

Failure mode: Metrics are collected because they are available, not decision-relevant.

Control: Start from the adoption or remediation decision.

Acceptance evidence: Evaluation charter

5.2 Golden task suite

Mechanism

1. Sample real work
2. Freeze fixture
3. Run candidate
4. Compare outcome

task	fixture	run	comparison
Required state field for sample real work	Required state field for freeze fixture	Required state field for run candidate	Required state field for compare outcome

Evidence and control

MEASURE

Representative tasks passing repeatable checks

Failure mode: A single easy demo overstates capability.

Control: Use a stratified task set including edge and failure cases.

Acceptance evidence: Versioned fixtures and run results

5.3 Functional correctness

Mechanism

1. Run tests
2. Inspect behavior
3. Review diff
4. Re-test repair

tests	behavior	diff	retest
Required state field for run tests	Required state field for inspect behavior	Required state field for review diff	Required state field for re-test repair

Evidence and control

MEASURE

Passed required checks / total required checks

Failure mode: A passing unit suite misses visible or integration defects.

Control: Combine automated, browser and human review criteria.

Acceptance evidence: Test report, observed behavior and diff

5.4 Reliability and variance

Mechanism

1. Repeat task
2. Capture outcome
3. Measure variance
4. Investigate cause

repetitions	outcomes	variance	cause
Required state field for repeat task	Required state field for capture outcome	Required state field for measure variance	Required state field for investigate cause

Evidence and control

MEASURE

Success rate and outcome variance over repeated runs

Failure mode: One successful run is treated as stable performance.

Control: Repeat representative tasks under controlled conditions.

Acceptance evidence: Run series and variance summary

5.5 Latency and throughput

Mechanism

1. Timestamp start
2. Record stages
3. Locate bottleneck
4. Re-test change

start	stages	bottleneck	result
Required state field for timestamp start	Required state field for record stages	Required state field for locate bottleneck	Required state field for re-test change

Evidence and control

MEASURE

Median completion time and useful tasks per hour

Failure mode: Faster output increases rework or reviewer load.

Control: Pair speed measures with defect and review effort.

Acceptance evidence: Stage timings and quality-adjusted throughput

5.6 Cost and resource efficiency

Mechanism

1. Count usage
2. Value reviewer time
3. Add rework
4. Compare baseline

usage	review_time	rework	baseline
Required state field for count usage	Required state field for value reviewer time	Required state field for add rework	Required state field for compare baseline

Evidence and control

MEASURE

Total cost per accepted task

Failure mode: Token or subscription cost is measured without human correction time.

Control: Use fully loaded cost per accepted outcome.

Acceptance evidence: Cost model and sensitivity table

5.7 Security and privacy evaluation

Mechanism

1. Map data
2. Probe boundary
3. Inspect logs
4. Record residual risk

data	boundary	logs	risk
Required state field for map data	Required state field for probe boundary	Required state field for inspect logs	Required state field for record residual risk

Evidence and control

MEASURE

Material risks with tested controls and owners

Failure mode: Secrets, personal data or proprietary code leak through tools or evidence.

Control: Use least privilege, redaction and explicit retention rules.

Acceptance evidence: Threat model, probe results and risk owner

5.8 Comparative experiment

Mechanism

1. Define baseline
2. Hold task constant
3. Run alternatives
4. Interpret trade-off

baseline	task	alternatives	tradeoff
Required state field for define baseline	Required state field for hold task constant	Required state field for run alternatives	Required state field for interpret trade-off

Evidence and control

MEASURE

Paired results across identical tasks and criteria

Failure mode: Different tasks or reviewers invalidate the comparison.

Control: Use controlled paired trials and document confounders.

Acceptance evidence: Comparison table and caveat register

5.9 Failure taxonomy and repair

Mechanism

1. Classify failure
2. Find locus
3. Apply scoped repair
4. Re-run affected checks

class	locus	repair	retest
Required state field for classify failure	Required state field for find locus	Required state field for apply scoped repair	Required state field for re-run affected checks

Evidence and control

MEASURE

Repairs that close the failed criterion without regression

Failure mode: A broad retry hides the cause and changes unrelated behavior.

Control: Repair the smallest causal layer and re-test scope plus regression set.

Acceptance evidence: Failure record, patch and re-test result

5.10 Adoption readiness

Mechanism

1. Aggregate evidence
2. Score controls
3. Identify gaps
4. Choose posture

evidence	control_score	gaps	posture
Required state field for aggregate evidence	Required state field for score controls	Required state field for identify gaps	Required state field for choose posture

Evidence and control

MEASURE

Readiness score supported by retrievable evidence

Failure mode: A recommendation ignores operating ownership and maintenance cost.

Control: Evaluate people, process, technology and governance together.

Acceptance evidence: Readiness scorecard and gap owners

5.11 Governed recommendation

Mechanism

1. Summarise benefit
2. State limits
3. Set controls
4. Approve next step

benefit	limits	controls	decision
Required state field for summarise benefit	Required state field for state limits	Required state field for set controls	Required state field for approve next step

Evidence and control

MEASURE

Recommendation accepted with conditions and review date

Failure mode: A binary adopt/reject decision hides scope and safeguards.

Control: Recommend a bounded posture: pilot, expand, hold or stop.

Acceptance evidence: Signed recommendation with conditions

6. Hands-on Lab Guide

Each lab is self-contained and includes a detailed README/PDF, lab-local AGENTS.md, bounded prompt, runnable starter and solution repository, verifier, expected output and evidence checklist.

6.1 Lab 01: Repository Reconnaissance and Baseline

OBJECTIVE

Inspect a TaskBoard repository, identify its instruction and test boundaries, and record a measurable baseline before any edit.

Folder contents

- labs/lab-01-repository-reconnaissance/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Open repo	Inputs, changed state, permissions, observable status and retained verification evidence
2	Read rules	Inputs, changed state, permissions,

		observable status and retained verification evidence
3	Map files	Inputs, changed state, permissions, observable status and retained verification evidence
4	Run tests	Inputs, changed state, permissions, observable status and retained verification evidence
5	Measure baseline	Inputs, changed state, permissions, observable status and retained verification evidence
6	Decide scope	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Inspect only. Map applicable instructions, runtime, tests and likely change files. Do not edit. Return evidence for every conclusion.

Acceptance check

PASS ONLY WHEN

The report names the applicable instruction chain, test command, baseline result and scoped target files.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.2 Lab 02: Prompt-to-Plan Feature Delivery

OBJECTIVE

Turn an outcome contract into a bounded plan, implement one TaskBoard feature and verify the changed behavior.

Folder contents

- labs/lab-02-prompt-plan-feature/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	State goal	Inputs, changed state, permissions, observable status and retained verification evidence
2	Inspect code	Inputs, changed state, permissions, observable status and retained verification evidence
3	Write plan	Inputs, changed state, permissions, observable status and retained verification evidence
4	Apply patch	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Run checks	Inputs, changed state, permissions, observable status and retained verification evidence
6	Review diff	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Add priority filtering to TaskBoard. Preserve the CLI contract, add tests, and finish only when the focused and regression checks pass.

Acceptance check

PASS ONLY WHEN

The diff is scoped, the new behavior is tested, all supplied tests pass and the final summary cites evidence.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.3 Lab 03: Scoped AGENTS.md Hierarchy

OBJECTIVE

Design root and nested instructions, then verify that a task applies the nearest rule without conflicting with repository policy.

Folder contents

- labs/lab-03-agents-hierarchy/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Read root	Inputs, changed state, permissions, observable status and retained verification evidence
2	Read nested	Inputs, changed state, permissions, observable status and retained verification evidence
3	Resolve scope	Inputs, changed state, permissions, observable status and retained verification evidence
4	Execute task	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Inspect diff	Inputs, changed state, permissions, observable status and retained verification evidence
6	Record trace	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Modify the API fixture under its nearest instructions. Explain which rules apply before editing and verify compliance after the patch.

Acceptance check

PASS ONLY WHEN

The trace distinguishes root and nested rules and the patch complies with both in the correct precedence.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.4 Lab 04: Sandbox and Approval Experiment

OBJECTIVE

Compare read-only, workspace-write and approval boundaries using safe commands and an explicit consequence matrix.

Folder contents

- labs/lab-04-sandbox-approval/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Classify action	Inputs, changed state, permissions, observable status and retained verification evidence
2	Set sandbox	Inputs, changed state, permissions, observable status and retained verification evidence
3	Run probe	Inputs, changed state, permissions, observable status and retained verification evidence
4	Observe block	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Request approval	Inputs, changed state, permissions, observable status and retained verification evidence
6	Recommend policy	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Probe only the supplied lab paths. Do not alter files outside the lab. Explain every boundary and record whether the action was allowed, blocked or approved.

Acceptance check

PASS ONLY WHEN

The log contains safe probes for three permission cases and a least-privilege recommendation supported by results.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.5 Lab 05: Parallel Worktrees without Collision

OBJECTIVE

Split two independent TaskBoard changes into isolated worktrees and integrate them in a controlled order.

Folder contents

- labs/lab-05-parallel-worktrees/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Find seams	Inputs, changed state, permissions, observable status and retained verification evidence
2	Create branches	Inputs, changed state, permissions, observable status and retained verification evidence
3	Create worktrees	Inputs, changed state, permissions, observable status and retained verification evidence
4	Run tasks	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Test commits	Inputs, changed state, permissions, observable status and retained verification evidence
6	Integrate	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Deliver documentation and test-fixture changes in separate worktrees. Do not edit the same file. Return commit SHAs and integration order.

Acceptance check

PASS ONLY WHEN

The worktrees use distinct paths and branches, produce separate commits, and integrate with the full test suite passing.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.6 Lab 06: Browser QA with Playwright

OBJECTIVE

Verify a local TaskBoard dashboard through a real browser and convert one visible defect into reproducible evidence.

Folder contents

- labs/lab-06-browser-qa/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Serve app	Inputs, changed state, permissions, observable status and retained verification evidence
2	Open page	Inputs, changed state, permissions, observable status and retained verification evidence
3	Exercise flow	Inputs, changed state, permissions, observable status and retained verification evidence
4	Capture defect	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Repair UI	Inputs, changed state, permissions, observable status and retained verification evidence
6	Re-test	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Test the priority filter at desktop and mobile viewports. Capture the failing state, repair only the causal UI layer, and repeat the path.

Acceptance check

PASS ONLY WHEN

The evidence includes pre-repair failure, scoped patch and passing desktop/mobile re-test.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.7 Lab 07: Build and Evaluate a Codex Skill

OBJECTIVE

Package a test-triage workflow as a reusable skill and measure trigger precision with positive and near-miss prompts.

Folder contents

- labs/lab-07-reusable-skill/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Define trigger	Inputs, changed state, permissions, observable status and retained verification evidence
2	Write skill	Inputs, changed state, permissions, observable status and retained verification evidence
3	Add template	Inputs, changed state, permissions, observable status and retained verification evidence
4	Run positive	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Run near-miss	Inputs, changed state, permissions, observable status and retained verification evidence
6	Refine	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Create a narrowly triggered skill for Python unittest failure triage. Include evidence requirements and test one prompt that must not trigger it.

Acceptance check

PASS ONLY WHEN

The skill passes structural checks and its evaluation shows correct trigger and near-miss behavior.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.8 Lab 08: Headless Codex Exec for CI

OBJECTIVE

Design a non-interactive Codex run that emits machine-readable events and fails safely when acceptance checks fail.

Folder contents

- labs/lab-08-headless-exec/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Define task	Inputs, changed state, permissions, observable status and retained verification evidence
2	Choose JSON	Inputs, changed state, permissions, observable status and retained verification evidence
3	Run exec	Inputs, changed state, permissions, observable status and retained verification evidence
4	Parse events	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Check exit	Inputs, changed state, permissions, observable status and retained verification evidence
6	Archive evidence	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Run the supplied repair task non-interactively. Emit JSON events, preserve stderr, and return non-zero if the required tests do not pass.

Acceptance check

PASS ONLY WHEN

The wrapper avoids embedded secrets, records structured events and distinguishes success from failure by exit status.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.9 Lab 09: Golden-Task Evaluation Harness

OBJECTIVE

Run a repeatable golden-task suite and calculate correctness, reliability, latency and cost-per-accepted-task.

Folder contents

- labs/lab-09-evaluation-harness/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Load fixtures	Inputs, changed state, permissions, observable status and retained verification evidence
2	Run candidate	Inputs, changed state, permissions, observable status and retained verification evidence
3	Score criteria	Inputs, changed state, permissions, observable status and retained verification evidence
4	Repeat trials	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Calculate metrics	Inputs, changed state, permissions, observable status and retained verification evidence
6	Diagnose failure	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Evaluate the supplied candidate outputs against frozen fixtures. Do not adjust the expected results. Report uncertainty and failed criteria.

Acceptance check

PASS ONLY WHEN

The report is reproducible, includes at least four decision-relevant metrics and links each failure to a fixture.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

6.10 Lab 10: Capstone: Governed Codex Adoption

OBJECTIVE

Deliver a bounded TaskBoard improvement, evaluate it against the baseline and make a pilot, expand, hold or stop recommendation.

Folder contents

- labs/lab-10-capstone-governed-adoption/README.md — detailed procedure
- README.pdf — detailed lab procedure and troubleshooting
- AGENTS.md and prompt.md — local operating rule and bounded task contract
- starter/ and solution/ — runnable Python TaskBoard states with tests
- scripts/verify_lab.py, solution/expected-output.json and evidence/checklist.pdf

Detailed procedure

1. Open the individual lab folder and read README.pdf, AGENTS.md, prompt.md and evidence/checklist.pdf before starting.
2. Run ``python3 scripts/verify_lab.py`` from the lab folder; confirm that the protected reference solution passes.
3. Copy starter/ to a temporary learner working directory inside the lab. Preserve the original starter for comparison.
4. Ask Codex to inspect first. Record the applicable instruction hierarchy, runtime, tests and proposed file scope.
5. Provide the bounded prompt from prompt.md. Check that the plan serves the objective and respects every protected boundary.
6. Approve only exact, safe actions needed for the task. Do not grant broad filesystem, network or destructive access.
7. Review every changed file in the diff. Reject unrelated edits, embedded credentials and unsupported assumptions.
8. Run the focused tests followed by the lab verifier. Record commands, outputs and exit status.
9. Trigger the documented controlled failure, observe its evidence, then restore the accepted state.
10. Complete the evidence checklist and make a PASS, REPAIR or STOP decision against the acceptance rule.

Lab-specific workflow checks

Sequence	Workflow stage	What to inspect
1	Frame problem	Inputs, changed state, permissions, observable status and retained verification evidence
2	Configure harness	Inputs, changed state, permissions, observable status and retained verification evidence
3	Implement change	Inputs, changed state, permissions, observable status and retained verification evidence
4	Verify outcome	Inputs, changed state, permissions,

		observable status and retained verification evidence
5	Compare baseline	Inputs, changed state, permissions, observable status and retained verification evidence
6	Recommend posture	Inputs, changed state, permissions, observable status and retained verification evidence

Prompt contract

Use the full Codex workflow to improve TaskBoard. Preserve user changes, verify every acceptance criterion, compare with baseline, and issue a conditional adoption recommendation.

Acceptance check

PASS ONLY WHEN

The solution works, the evidence pack covers correctness, efficiency and risk, and the recommendation states limits, controls, owner and review date.

Troubleshooting

Observed issue	Likely cause	Remedial action
Codex edits before explaining scope	Prompt omitted inspect-first and finish conditions	Stop, restore the starter copy and re-run with the complete bounded contract.
Command is blocked	Sandbox or approval policy is narrower than the action	Reduce scope or request a precise approval that names the target and consequence.
Output is plausible but unsupported	Finish condition omitted observable checks	Require the diff, test output and affected-file list before accepting completion.
Tests cannot import taskboard	Test command ran from the wrong folder	Run the documented unittest command from starter/ or solution/.

7. Codex Control and Evidence Contract

A Codex task is complete only when the requested state exists and the required checks demonstrate it. Preserve the task contract, applicable instruction chain, starting repository state, reviewed diff, command results, test or browser evidence and the named human decision at any consequential boundary.

DEFAULT EXECUTION POSTURE

Start with the narrowest workable permissions, preserve user changes, keep external and destructive actions behind explicit approval, and reject completion claims that are not supported by retrievable evidence.

8. References

- **Official course page:** <https://www.tertiarycourses.com.sg/agent-ai-applications-with-codex.html> — course identity, duration, learning outcomes and delivery context
- **OpenAI Codex learning hub:** <https://developers.openai.com/learn/codex> — current Codex workflows, guides and reference entry points
- **Codex cloud documentation:** <https://learn.chatgpt.com/docs/cloud> — cloud environments, parallel tasks and review workflow
- **OpenAI Academy: Codex:** <https://openai.com/academy/codex/> — official learning resources and demonstrations
- **OpenAI Codex product:** <https://openai.com/codex/> — Codex product surfaces and capabilities
- **OpenAI Codex repository:** <https://github.com/openai/codex> — CLI installation, repository guidance and source reference
- **AIFLOXUM Codex guide:** <https://www.aifloxiu.com/blog/openai-codex-tutorial-complete-guide-2026> — supplementary workflow coverage
- **QCode Codex IDE guide:** <https://docs.qcode.cc/en/docs/ide/codex> — supplementary IDE integration perspective
- **DataCamp Codex tutorial:** <https://www.datacamp.com/tutorial/openai-codex> — supplementary cloud-task examples
- **YouMind Codex tutorial:** <https://youmind.com/landing/x-viral-articles/master-codex-ai-features-tutorial> — supplementary feature overview
- **AirMore Codex beginners review:** <https://airmore.ai/ai-review/codex-tutorial-beginners> — supplementary beginner perspective
- **Gauraw Codex tutorial:** <https://www.gauraw.com/openai-codex-tutorial/> — supplementary workflow and troubleshooting coverage
- **DeepWiki Codex interfaces:** <https://deepwiki.com/openai/codex/4-user-interfaces> — supplementary interface taxonomy

Course repository: <https://github.com/tertiarycourses/C695-Agent-AI-Applications-with-Codex>

Learning platform: <https://lms.tertiaryinfotech.com>